LÖSUNGEN · LEHREREXEMPLAR

W5·2 – Der Wald: Suchbäume – Lösungen

Zum Arbeitsblatt `AQ_W5_AB2_Suchbaeume.docx` · IDE: `AQ_W5_L5_Wald_LSG.txt` (Klasse Suchbaum, L54SuchbaumTest) und `AQ_W5_Baumzeichner_LSG.txt` (ItemDatenbank) · alle Bäume, Suchwege, Rückgaben, Rekursionstabellen und Laufzeitfehler mit Java (javac 21) und einem exakten Nachbau von BinTree (Object-basiert, gleiche Exceptions) erzeugt; die abgedruckten Methoden liefern dieselben Ergebnisse wie die Musterlösung · Bäume in Klammerform: `x(links,rechts)`, „-“ = leerer Teilbaum

Teil 1 (DS 34)

Aufgabe 1 – Echt oder gefälscht? (8 BE)

Baum	Suchbaum?	Begründung	Inorder
A	ja	Für jeden Knoten links nur Kleinere, rechts nur Größere.	20 30 40 50 60 70 80
B	nein	55 steht im **linken** Teilbaum von 50, ist aber größer als 50. (Zu ihrem Elternknoten 30 passt die 55!)	20 30 55 50 70
C	nein	75 ist linkes Kind von 70, aber größer als 70.	20 30 40 50 75 70 80
D	nein	40 steht im rechten Teilbaum der Wurzel 40 – nicht **größer** als 40 (Duplikat). Jeder Knoten passt zu seinen Kindern.	10 20 30 40 40 60 70

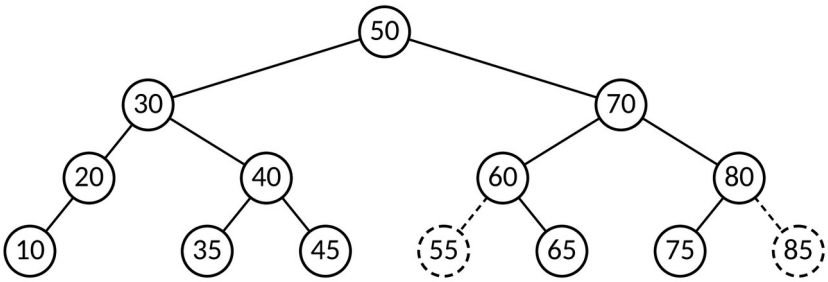
a) je 1 BE (mit Java geprüft: istBST A wahr, B falsch, C falsch, D falsch). **b)** Inorder B `20 30 55 50 70` (55 vor 50), D `10 20 30 40 40 60 70` (40 doppelt). Ein Baum ist genau dann ein Suchbaum, wenn seine Inorder **streng** aufsteigend ist – jedes Element echt größer als sein Vorgänger (2 BE). **c)** 35, 40, 45. Jede Nummer muss **größer als 30** sein (der Teilbaum liegt rechts von 30) und **kleiner als 50** (er liegt im linken Teilbaum der Wurzel 50). Die Grenzen legen also Vorfahren fest, nicht nur der Elternknoten – Vorbereitung auf istBST mit Grenzen in Aufgabe 10 (2 BE).

Aufgabe 2 – Suchen mit Zählwerk (7 BE)

x	Suchweg	untersuchte Knoten	suche(b, x)
65	50, 70, 60, 65	4	wahr
33	50, 30, 40, 35 → leer	4	falsch
80	50, 70, 80	3	wahr
50	50	1	wahr
99	50, 70, 80 → leer	3	falsch
5	50, 30, 20, 10 → leer	4	falsch

a) 3 BE (je zwei Zeilen 1 BE). **b)** Summe der untersuchten Knoten über alle 12 Items: 37, Durchschnitt 37 / 12 ≈ **3,08**. Höchstens werden **4** Knoten untersucht (für alle Nummern 0 bis 100 geprüft) – das ist die **Höhe** 4 des Baums: Die Suche geht pro Ebene genau einen Knoten tiefer und endet spätestens in einem Blatt bzw. dem leeren Teilbaum darunter (3 BE). **c)** Linear: im Durchschnitt (1 + 2 + … + 12) / 12 = 6,5 Vergleiche, wenn das Item vorhanden ist, im schlechtesten Fall 12 (bei einer fehlenden Nummer immer 12). Der Suchbaum braucht etwa halb so viele, höchstens 4 (1 BE).

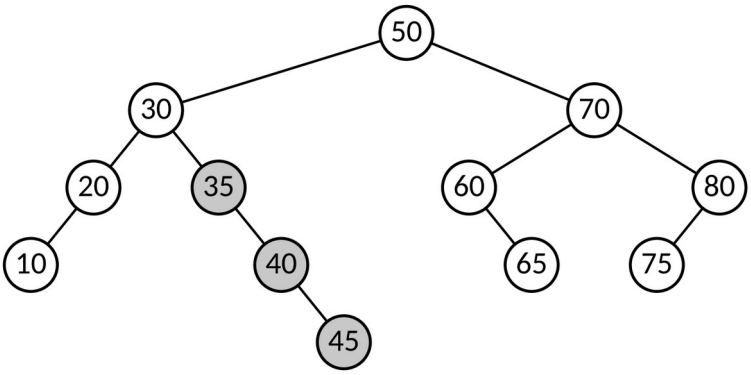
Aufgabe 3 – Neue Items für die Datenbank (8 BE)



Item-Datenbank nach dem Einfügen von 55 und 85 (gestrichelt: neue Knoten)

a) 55: Suchweg 50, 70, 60 → leer → **links von 60**. 85: Suchweg 50, 70, 80 → leer → **rechts von 80**. Klammerform: `50(30(20(10,-),40(35,45)),70(60(55,65),80(75,85)))` (2 BE). **b)** Die Suche nach 30 endet bei der 30 (x == wert); es wird nichts eingefügt, der Baum bleibt unverändert – Duplikate werden ignoriert (1 BE).

c) (1) **gleicher Baum** (geprüft). (2) **anderer Baum**: 35 kommt vor 40 und wird deshalb rechtes Kind von 30; 40 und 45 hängen als rechte Kette darunter. Höhe **5**, Klammerform `50(30(20(10,-),35(-,40(-,45))),70(60(-,65),80(75,-)))` (3 BE).



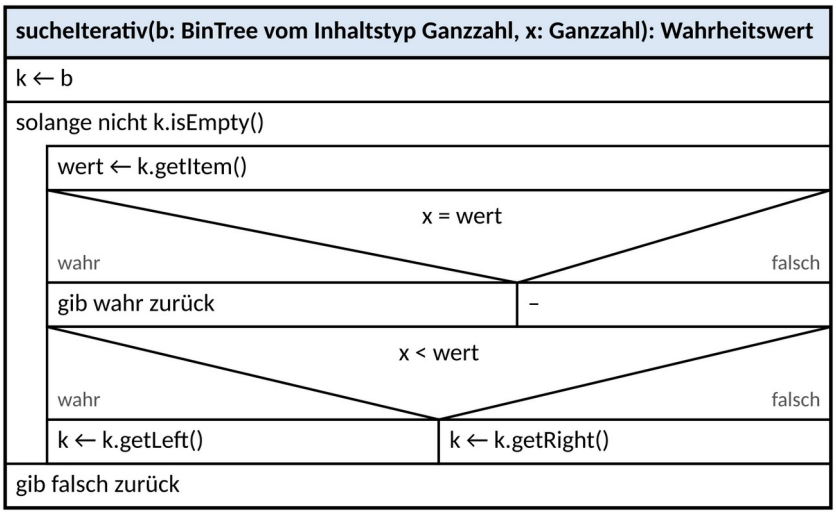
Baum zur Reihenfolge (2) – grau: der Ast, der anders aussieht als in Abbildung 1

d) Es entsteht genau der Baum aus Abbildung 1, wenn **jeder Wert erst nach allen seinen Vorfahren** (insbesondere nach seinem Elternknoten) eingefügt wird; die Reihenfolge zwischen verschiedenen Ästen ist egal. Also: zuerst 50, 30 und 70 vor ihren Nachfahren usw. (2 BE). Per Programm an 4000 Reihenfolgen geprüft (2000 zufällige, 2000 regelkonforme): Die Regel trifft in allen Fällen genau zu.

Aufgabe 4 – Struktogramm ↔ Java (8 BE)

```
public static Item finde(BinTree b, int nummer) {
    if (b.isEmpty()) {
        return null;                        // Nummer nicht vorhanden
    }
    Item hier = (Item) b.getItem();         // Cast: Object -> Item
    if (nummer == hier.getNummer()) {
        return hier;
    }
    if (nummer < hier.getNummer()) {
        return finde(b.getLeft(), nummer);
    }
    return finde(b.getRight(), nummer);
}
```

a) 4 BE: Kopf mit Typen (`Item` als Rückgabe) · Basisfall mit `return null` · Cast `(Item) b.getItem()` · Vergleich über `getNummer()` · Rückgabe der rekursiven Aufrufe (nicht nur aufrufen!). Gleichwertig: Zwischenvariable `int n = hier.getNummer();`. **b)** `finde(b, 65).beschreibung()` liefert **"#65 Bogen"**, `finde(b, 33)` liefert **null** (1 BE).

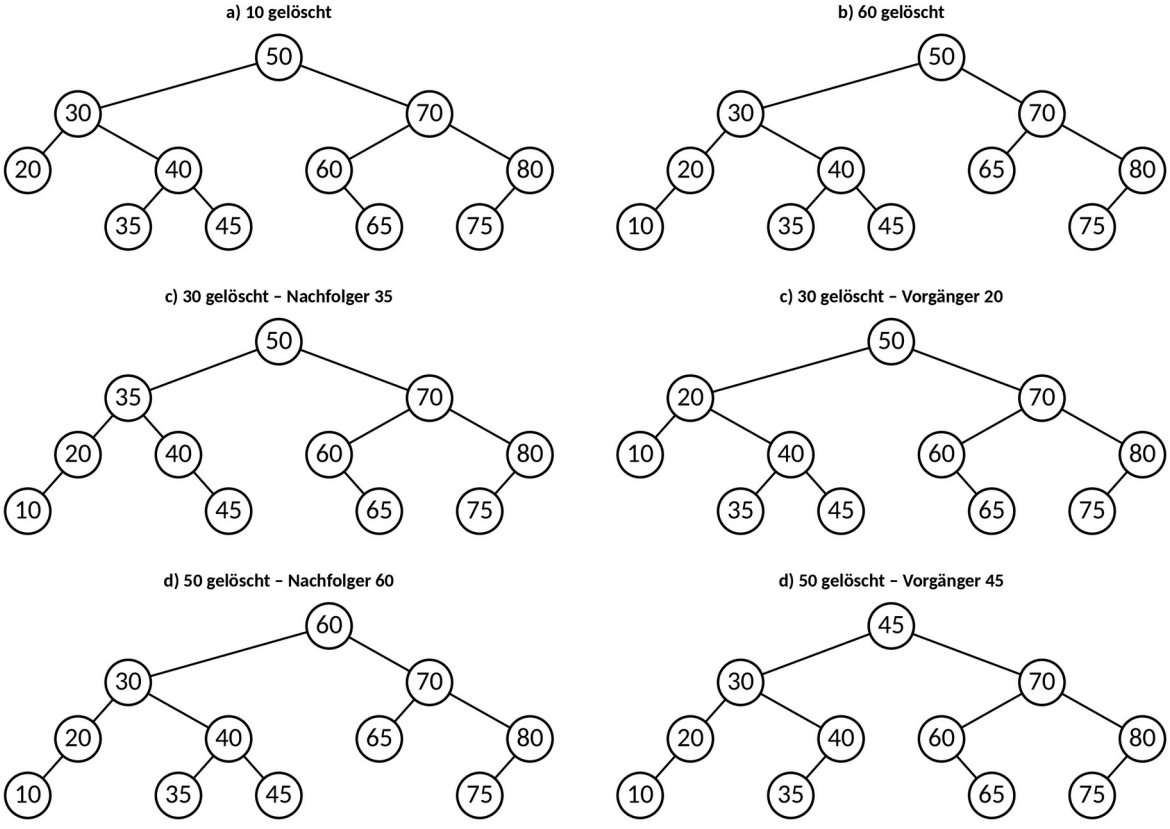


c) 3 BE: Kopf mit Signatur · Zuweisung k ← b · kopfgesteuerte Schleife „solange nicht k.isEmpty()“ · zwei Verzweigungen im Schleifenrumpf · „gib … zurück“ im Rumpf und nach der Schleife. Ohne Cast (sprachunabhängig). **d)** Papier-Compiler: Musterlösung wie Infoteil 2 bzw. `AQ_W5_L5_Wald_LSG.txt`; die vier genannten Basis-Tests sind mit der Musterlösung grün. Häufige Fehler: zweiter rekursiver Aufruf bei suche (basis_suche_nurEinWeg wird rot), `b = new BinTree(x)` statt `b.setItem(x)` (vgl. Aufgabe 6 A), fehlender Cast.

Infoteil 3 – Kontrollwerte Löschen am beispielBST

4 (Blatt) → `8(3(1,6(-,7)),10(-,14(13,-)))` · 14 (ein Kind) → `8(3(1,6(4,7)),10(-,13))` · 6 (zwei Kinder) mit Nachfolger 7 → `8(3(1,7(4,-)),10(-,14(13,-)))`, mit Vorgänger 4 → `8(3(1,4(-,7)),10(-,14(13,-)))`. Identisch mit `werte_w5.txt` und der Warm-up-Karte W5-12.

Aufgabe 5 – Ein Item verschwindet (10 BE)



Die Item-Datenbank nach den Löschungen (jeweils aus der ursprünglichen Datenbank)

a) 10 ist ein **Blatt** (Fall 1): Es wird entfernt, 20 hat danach keine Kinder. 60 hat **genau ein Kind** (65, Fall 2): 65 rückt an die Stelle von 60 und wird linkes Kind von 70 (2 BE).

b) 30 hat zwei Kinder (Fall 3). Nachfolger = kleinster Wert im rechten Teilbaum: einmal rechts (40), dann immer links → **35**; 35 ersetzt 30 und wird an seiner alten Stelle als Blatt entfernt: `50(35(20(10,-),40(-,45)),70(60(-,65),80(75,-)))`. Vorgänger = größter Wert im linken Teilbaum: einmal links (20), dann immer rechts → **20** (20 hat kein rechtes Kind); 20 ersetzt 30, an der alten Stelle rückt 10 nach (Fall 2): `50(20(10,40(35,45)),70(60(-,65),80(75,-)))` (3 BE).

c) Nachfolger der Wurzel 50: rechts 70, dann links 60, weiter links ist nichts → **60**. 60 ersetzt 50; an der alten Stelle von 60 rückt 65 nach (Fall 2): `60(30(20(10,-),40(35,45)),70(65,80(75,-)))`. Inorder-Probe: `10 20 30 35 40 45 60 65 70 75 80` – die alte Inorder ohne 50 (3 BE). Alternative mit Vorgänger 45: `45(30(20(10,-),40(35,-)),70(60(-,65),80(75,-)))`.

d) Der Nachfolger ist das Minimum des rechten Teilbaums: Man geht von dort so lange nach links, bis **links nichts mehr** ist. Hätte der Nachfolger ein linkes Kind, wäre dieses kleiner und läge ebenfalls im rechten Teilbaum – Widerspruch. Er hat also höchstens ein (rechtes) Kind; ihn an seiner alten Stelle zu entfernen ist Fall 1 oder Fall 2. Die Ordnung bleibt erhalten, weil der Nachfolger größer als alles im linken und kleiner als alles übrige im rechten Teilbaum ist (2 BE).

Aufgabe 6 – Fehlerjagd in der Datenbank (9 BE)

a) (A) Die Item-Datenbank bleibt nach `einfuegen(b, 55)` **unverändert**, ein leerer Baum bleibt **leer** (isEmpty() = wahr). Ursache Zeile 3: `b = new BinTree(x)` ändert nur die **lokale Parametervariable** b – sie zeigt danach auf ein neues Blatt, das niemand kennt; der leere Teilbaum im eigentlichen Baum bleibt leer. Korrektur: `b.setItem(x);` – setItem verändert das Objekt selbst und macht den leeren Teilbaum zum Blatt (3 BE).

Baum	fehlerhaftes minimum	richtig
Item-Datenbank	10	10
beispielBST	1	1
entartet() (1 bis 5)	−1	1
5(2(-,3),9)	−1	2

b) (B) Fehler Zeile 5: Das Minimum ist nicht das erste **Blatt** auf dem Weg nach links, sondern der Knoten, dessen **linker Teilbaum leer** ist. Bei 5(2(-,3),9) hat 2 ein rechtes Kind, ist also kein Blatt; die Methode geht nach links in den leeren Baum und liefert −1. Korrektur: `if (b.getLeft().isEmpty())` statt `if (b.isLeaf())`. In der Item-Datenbank (und im beispielBST) ist das Minimum zufällig ein Blatt – Tests nur mit diesem Baum bleiben grün. Deshalb prüft `basis_minimum` auch den entarteten Baum und 5(2(-,3),9) (3 BE).

c) (C) x = 50: **wahr** (richtig) · x = 65: **falsch** (falsch – bei der 65 ist k ein Blatt, die Schleife endet, ohne die 65 zu prüfen) · x = 99: **Laufzeitfehler (IllegalStateException)** (nach 80 ist k der leere rechte Teilbaum, isLeaf auf leerem Baum) · leerer Baum: **Laufzeitfehler (IllegalStateException)**. Korrektur Zeile 3: `while (!k.isEmpty())` (3 BE).

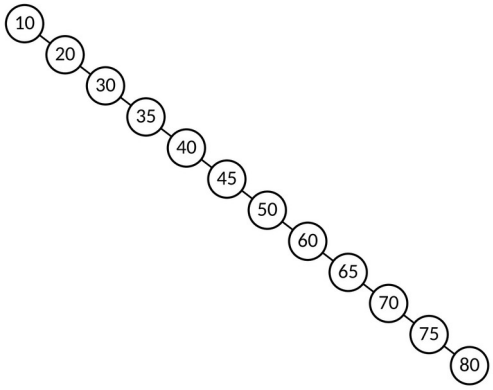
Aufgabe 7 – Level 5.4 Basis und Erweiterung

Musterlösungen und Hinweise zu häufigen Fehlern: `AQ_W5_L5_Wald_LSG.txt`, Klasse Suchbaum (L54SuchbaumTest: 11 Tests, mit der Musterlösung alle grün). Kontrollwerte: vergleiche(beispielBST, 7) = 4, vergleiche(beispielBST, 5) = 4, vergleiche(entartet, 5) = 5; baueAus mit Reihenfolge (2) aus Aufgabe 3 ergibt `50(30(20(10,-),35(-,40(-,45))),70(60(-,65),80(75,-)))`. Baumzeichner-ItemDatenbank: inventar() liefert `#10 Brot; #20 Seil; #30 Fackel; #35 Karte; #40 Schlüssel; #45 Laterne; #50 Heiltrank; #60 Schild; #65 Bogen; #70 Schwert; #75 Kompass; #80 Drachenei;`.

Teil 2 (DS 35)

Aufgabe 8 – Die Wäscheleine (7 BE)

a) Jede neue Nummer ist größer als alle bisherigen und wird rechts angehängt: eine **rechte Kette** (entarteter Baum) 10 → 20 → … → 80, Höhe **12**. Suche nach jedem Item einmal: 1 + 2 + … + 12 = 78 untersuchte Knoten, Durchschnitt **6,5** – wie die lineare Suche und gut doppelt so viel wie in Abbildung 1 (3,08) (2 BE).



n	7	15	63	127	255	1.000	1.023	1.000.000
sortiert eingefügt	7	15	63	127	255	1.000	1.023	1.000.000
kleinstmögliche Höhe	3	4	6	7	8	10	10	20

b) 3 BE. Kleinstmögliche Höhe = kleinstes k mit 2^k − 1 ≥ n, berechnet mit ausgeglichenAus; bis n = 1023 auch die größte Anzahl untersuchter Knoten per vergleiche nachgezählt (stimmt jeweils mit der Höhe überein). Für 1 000 000 wurde die Kette nicht gebaut (Stapelüberlauf beim rekursiven Einfügen) – ihre Höhe ist n.

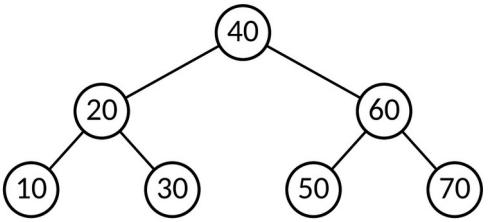
c) Ein vollständiger Baum mit k Ebenen hat 2^k − 1 Knoten. Eine weitere Ebene bietet Platz für 2^k neue Knoten – fast eine Verdopplung: 2^(k+1) − 1 = 2 · (2^k − 1) + 1. Doppelt so viele Items brauchen also nur **eine** Ebene mehr, d. h. **einen** Vergleich mehr. Bei 1 000 000 Items: ausgeglichen höchstens **20** untersuchte Knoten, entartet bis zu 1 000 000. Die Item-Datenbank lohnt sich nur, wenn der Baum nicht entartet – z. B. Items nicht sortiert einfügen oder mit ausgeglichenAus neu aufbauen (2 BE).

Aufgabe 9 – Mitte nach oben (9 BE)

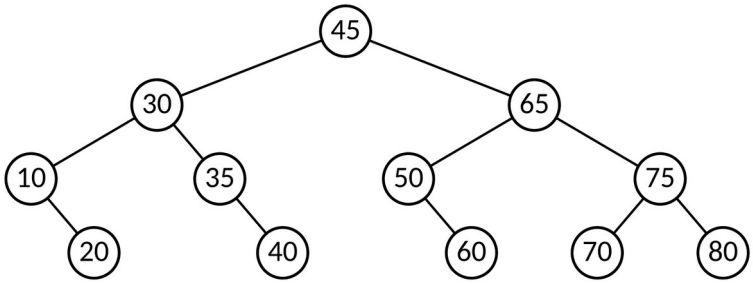
Rekursionstiefe	lo	hi	mitte	liefert
0	0	6	3	Knoten 40
1	0	2	1	Knoten 20
2	0	0	0	Knoten 10
3	0	-1	–	leerer Baum
3	1	0	–	leerer Baum
2	2	2	2	Knoten 30
3	2	1	–	leerer Baum
3	3	2	–	leerer Baum
1	4	6	5	Knoten 60
2	4	4	4	Knoten 50
3	4	3	–	leerer Baum
3	5	4	–	leerer Baum
2	6	6	6	Knoten 70

Rekursionstiefe	lo	hi	mitte	liefert
3	6	5	–	leerer Baum
3	7	6	–	leerer Baum

a) 4 BE: Reihenfolge des Beginns (erst der ganze linke Teilbaum, dann der rechte), mitte ganzzahlig, Basisfall lo > hi mit leerem Baum; Tiefe des Erstaufrufs 0.



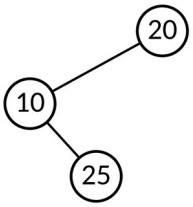
b) Baum `40(20(10,30),60(50,70))`, Höhe 3, **15 Aufrufe** = 2 · 7 + 1. Begründung: Jeder der n Werte wird in genau einem Aufruf zur Wurzel eines Teilbaums (n Aufrufe mit lo ≤ hi). Jeder dieser Aufrufe ruft zweimal auf; insgesamt gibt es also 1 (Erstaufruf) + 2n Aufrufe, davon n + 1 mit leerem Bereich (2 BE).



c) Baum `45(30(10(-,20),35(-,40)),65(50(-,60),75(70,80)))`, Höhe **4**, ein Suchbaum (istBST wahr). Suche nach 65: 45, 65 → **2** untersuchte Knoten. Die Summe über alle 12 Items ist 37 – genauso gut wie die ursprüngliche Datenbank, kleinstmögliche Höhe für 12 Werte (3 BE). Hinweis: mitte = (lo + hi) / 2 rundet ab; bei gerader Länge wird das **linke** der beiden mittleren Elemente Wurzel (hier 45 mit Index 5).

Aufgabe 10 – Die isBST-Falle (9 BE)

a) `nurKinder(F)` liefert **wahr**: 55 ist rechtes Kind von 45 und größer als 45. F ist trotzdem **kein** Suchbaum (istBST: falsch): 55 steht im **linken** Teilbaum der Wurzel 50, ist aber größer als 50; die Inorder `10 20 30 35 40 45 55 50 60 65 70 75 80` ist nicht sortiert (55 vor 50). Folge: `suche(F, 55)` geht den Weg 50, 70, 60 → leer und liefert **falsch**, obwohl 55 im Baum steht (3 BE).



b) z. B. `20(10(-,25),-)`: 25 ist rechtes Kind von 10 (passt), liegt aber im linken Teilbaum von 20 (zu groß). nurKinder wahr, istBST falsch (1 BE). Gleichwertig jeder Baum mit einem „Enkel“ auf der falschen Seite des Großelternknotens.

Rekursionstiefe	Knoten	min	max	Rückgabe
0	50	–	–	falsch
1	30	–	50	falsch
2	20	–	30	wahr
3	10	–	20	wahr
4	leer	–	10	wahr
4	leer	10	20	wahr

Rekursionstiefe	Knoten	min	max	Rückgabe
3	leer	20	30	wahr
2	40	30	50	falsch
3	35	30	40	wahr
4	leer	30	35	wahr
4	leer	35	40	wahr
3	45	40	50	falsch
4	leer	40	45	wahr
4	55	45	50	falsch (55 ≥ 50)

c) 4 BE. Die Grenzen werden nach unten weitergegeben: Nach links wird der Wert zur neuen **Obergrenze**, nach rechts zur neuen **Untergrenze**. Bei 55 (Grenzen 45 und 50) schlägt `wert >= max` zu. Wegen `&&` werden danach weder der übrige Teil noch der rechte Teilbaum der Wurzel (70 …) untersucht – 14 Aufrufe. Zum Vergleich: istBST der unveränderten Item-Datenbank braucht 25 Aufrufe (alle 12 Knoten und 13 leeren Teilbäume).

d) Alle Werte sind ≥ 0, also ist `wert <= -1` nie wahr – −1 als Untergrenze schränkt nichts ein. Als Obergrenze würde −1 dagegen jeden Wert verbieten (`wert >= -1` ist immer wahr); deshalb muss `max != -1` zuerst geprüft werden (1 BE). Gleichwertig: Integer.MAX_VALUE als Obergrenze (in der Online-IDE nicht geprüft).

Aufgabe 11 – Level 5.4 Experte

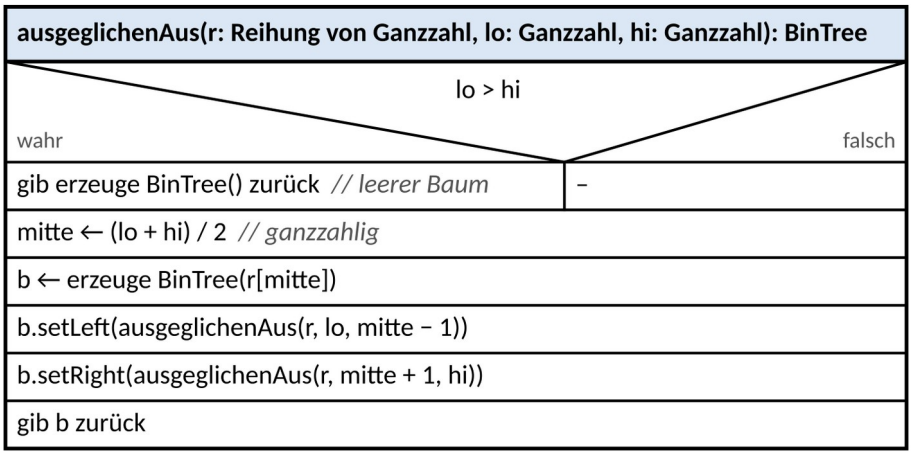
Musterlösungen: `AQ_W5_L5_Wald_LSG.txt` (istBST mit Grenzen wie oben, ausgeglichenAus wie Aufgabe 9, nachfolger). Häufige Fehler: istBST nur mit Kindern (experte_istBST wird mit den Fallen 8(3(1,9),10) und 8(3,10(5,14)) rot), ausgeglichenAus mit Bereich lo … mitte statt lo … mitte − 1 (Endlosrekursion), nachfolger liefert x selbst statt des nächstgrößeren Werts.

Für Schnelle – Löschen implementieren

```
public static void loesche(BinTree b, int x) {
    if (b.isEmpty()) {
        return;                              // x kommt nicht vor
    }
    int wert = (Integer) b.getItem();
    if (x < wert) {
        loesche(b.getLeft(), x);
    } else if (x > wert) {
        loesche(b.getRight(), x);
    } else if (b.isLeaf()) {                 // Fall 1: Blatt
        b.setEmpty();
    } else if (b.getLeft().isEmpty()) {      // Fall 2: nur rechtes Kind
        BinTree kind = b.getRight();
        b.setItem(kind.getItem());
        b.setLeft(kind.getLeft());
        b.setRight(kind.getRight());
    } else if (b.getRight().isEmpty()) {     // Fall 2: nur linkes Kind
        BinTree kind = b.getLeft();
        b.setItem(kind.getItem());
        b.setLeft(kind.getLeft());
        b.setRight(kind.getRight());
    } else {                                 // Fall 3: zwei Kinder
        int nf = minimum(b.getRight());      // Nachfolger
        b.setItem(nf);
        loesche(b.getRight(), nf);           // dort Fall 1 oder 2
    }
}

public static int minimum(BinTree b) {       // b nicht leer
    if (b.getLeft().isEmpty()) {
        return (Integer) b.getItem();
    }
    return minimum(b.getLeft());
}
```

Diese Methode hat alle Löschungen dieses Lösungsblatts (Nachfolger-Variante) erzeugt; nach jeder Löschung wurde istBST und die Knotenzahl 11 geprüft. Nachfolger in der Item-Datenbank: nachfolger(45) = 50, nachfolger(80) = −1, nachfolger(33) = 35.



Struktogramm zu ausgeglichenAus(r, lo, hi) (Für Schnelle)